A MATHLIB TURING-MACHINE STATEMENT FOR INTEGER MULTIPLICATION BELOW $n \log n$

DAN BROWN*

Abstract. We state the target of the `integer-mult-bounds-lean` formalization, integer multiplication in time $O(n(\log n)^{1-\kappa})$ for some $\kappa > 0$, using only definitions from Mathlib: the multi-stack machines `Turing.FinTM2`, the time-bounded computability predicate `Turing.TM2ComputableInTime`, and Mathlib's binary encodings of natural numbers. We prove in Lean that the repository's target proposition `Machine.EndToEnd` implies this statement. The proof compiles any multitape program of the repository's model into a Mathlib machine with linear time overhead. Applied to the repository's proved elementary multiplier, it gives an unconditional Mathlib machine that multiplies in quadratic time. All theorems depend only on the axioms `propext`, `Classical.choice` and `Quot.sound`.

1. Why a Mathlib statement

A formal theorem certifies its statement, so a reader who wants to know what has been proved must read every definition the statement mentions. The target `Machine.EndToEnd` mentions the repository's machine model: `Program`, `step`, `run`, the input convention `input`, the output convention `outputCorrect` with `wordTape` and `binaryValue`, and the time function `targetTime` with `lg`. These definitions are short and careful, and an auditor still has to check each of them: that a transition reads only scanned symbols, that heads move at most one cell, that the alphabet is finite, that the input and output conventions encode the intended problem, and that the time bound quantifies over all inputs of each length.

The statement below mentions only Mathlib definitions together with one abbreviation for Mathlib's own pair encoding. Mathlib's Turing machines are maintained and reviewed by the Mathlib community independently of this project, and their conventions are fixed in advance of any particular algorithm. An auditor of the final theorem reads the statement, runs `#print axioms`, and is done. The repository's machine model, the simulation, and every algorithmic construction become parts of the proof, which Lean's kernel checks. A defect in any of them would make the proof fail; it cannot make the statement weaker.

2. The statement

```
abbrev pairEnc : ℕ × ℕ → List (Bool ⊕ Bool) :=
  (encodingProd encodingNatBool encodingNatBool).encode

def MulInTime (κ : ℝ) : Prop :=
  ∃ M : TM2ComputableInTime pairEnc encodingNatBool.encode
      (fun p : ℕ × ℕ => p.1 * p.2),
    (fun n : ℕ => (M.time n : ℝ)) =O[atTop]
      fun n : ℕ => (n : ℝ) * Real.log n ^ (1 - κ)

def Statement : Prop := ∃ κ : ℝ, 0 < κ ∧ MulInTime κ
```

Date: October 11, 2026.

*Author of *The Da Vinci Code*.

The input of (a, b) is the least-significant-first binary word of a over the symbols `Sum.inl`, followed by that of b over `Sum.inr`; its length is $n = \lceil \log_2(a+1) \rceil + \lceil \log_2(b+1) \rceil$. The machine must halt with the least-significant-first binary word of ab, without leading zeroes, on its output stack, every other stack empty, and its local state reset. `TM2ComputableInTime` requires `M.time n` to bound the number of steps on every input of length n, so the bound is a worst-case bound.

Remark 1 (The model). A `FinTM2` has finitely many stacks, a finite set of labels, a finite local state and a finite input alphabet; one step executes one label's loop-free statement, a bounded amount of work. Two stacks simulate one tape with constant overhead, and a stack is a restricted tape, so running times on `FinTM2` and on multitape Turing machines agree up to constant factors. Mathlib's single-tape models `TM0` and `TM1` would not do: multiplication on one tape takes $\Omega(n^2)$ steps.

3. The bridge

Structured programs. `MathlibTM2.Stack` is a small imperative language over a finite family of stacks with straight-line actions, sequencing, conditionals, `while` loops and calls to a finite table of possibly recursive procedures. Its big-step semantics carries an exact step count. A compiler produces a `FinTM2` with one extra stack of return labels, and `Exec.toOutputs` turns an execution of the main procedure into `TM2OutputsInTime` with the same count plus two.

Simulation. Each tape of a `Machine.Program` with t tapes is stored as two stacks: the cells left of the head, nearest first, and the cells from the head rightwards. One loop iteration reads every scanned symbol, looks up the transition, and writes and moves on each tape; it takes $t + 5$ machine steps. The representation invariant and its preservation under a transition are `rep_step`.

Input and output. The bridge program splits the input into the bits of a and b, pads both to the common length $\max(\lceil \log_2(a+1) \rceil, \lceil \log_2(b+1) \rceil, 1)$ most significant bit first, writes $x\#y$ on tape 0, runs the simulation, drains tape 0 from left to right keeping its bit symbols and dropping leading zeroes onto the output stack, and empties every other stack. Each phase is a loop proved by induction on the list it consumes, with an exact step count.

Time. On inputs of length N the bridge takes at most

$$\mathrm{bridgeTime}(N) = (34t + 102)(N + 1) + (13t + 5) \max_{n \le N+1} \lceil \mathrm{total}(n) \rceil$$

steps, where $\mathrm{total}(n)$ bounds the multitape running time at padded length n.

4. Results

Theorem 2 (`tm2_of_runsWithin`)**.** *If a* `Machine.Program` *M with t tapes satisfies* `Assembly.RunsWithin M total`*, then there is a* `TM2ComputableInTime pairEnc encodingNatBool.encode` *machine for $(a, b) \mapsto ab$ whose time function is* bridgeTime*.*

Theorem 3 (`mulInTime_of_computesWithin`, `statement_of_endToEnd`)**.** *If $0 < \kappa \le 1$ and M satisfies* `ComputesWithin M` *κ, then* `MulInTime` *κ holds. In particular* `Machine.EndToEnd` *implies* `Statement`*.*

Theorem 4 (`mul_quadratic`)**.** *There is a* `TM2ComputableInTime pairEnc encodingNatBool.encode` *machine for $(a, b) \mapsto ab$ whose time function is $O(n^2)$.*

The last theorem instantiates the first with the repository's `ElementaryMultiply.runsWithin` and is unconditional. It exercises every component of the bridge on a proved program.

5. Verification

The files are in `IntegerMultBounds/MathlibTM2/`. From the repository root,

```
lake build IntegerMultBounds.MathlibTM2
lake env lean MathlibTM2Check.lean
```

The check file asserts with `#guard_msgs` that the four theorems above depend exactly on `propext`, `Classical.choice` and `Quot.sound`.

Email address: jdb1729@gmail.com